Version: 2026/03/17 10:51

# CSC1015F Assignment 5B (50 Marks)

Arrays

## Learning Objectives

By the end of this lesson, you should be able to:

- Be able to use while loops effectively in your programs.
- Create a list of items
- Perform the common operation on a list.
- Manipulate lists.
- Use the common List Functions.
- Use Lists in a Function.
- Understand what a multi-dimensional list is.
- Understand what a Dictionary is.

## Assignment Instructions

This assignment involves constructing Python programs that manipulate lists, dictionaries and strings. You can use the following built-in functions in Python: `map( ), math.sqrt( ), split( ), list( ), min( ), max( ).`

**NOTE** Your solutions to this assignment will be evaluated for correctness and for the following qualities:

- Documentation
  - Use of comments at the top of your code to identify program purpose, author and date.
  - Use of comments within your code to explain each non-obvious functional unit of code.
- General style/readability
  - The use of meaningful names for variables and functions.
- Algorithmic qualities
  - Efficiency, simplicity

These criteria will be manually assessed by a tutor and commented upon. In this assignment, up to 10 marks will be deducted for deficiencies.

## Question 1: Second Missing [15 marks]

Write a program called '`second_missing.py`' that takes in a list of integers (separated by spaces) which form a nearly consecutive sequence but with **two numbers missing**. The program must output both missing numbers in ascending order.

For example, given the following arbitrary list of numbers (separated by spaces), 10 12 13 14 16 17 18. The missing numbers are 11 and 15.

**Hints:**

- Assume the lowest and highest numbers given are the range of numbers.
- You may use only: `min()`, `max()`, `split()`, `list()`, `map()`, `print()`, `input()`, `int()`.
- You may *not* use built-in sorting functions.
- A missing number in the sequence is the difference between the expected sum of the numbers and the actual sum of the numbers.
- If one number is missing in the sequence or no number is missing, print "`Error: Expected exactly two missing numbers`" (see the sample IO below).

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter a sequence of space-separated numbers:
10 12 13 14 16 17 18
Missing numbers: 11, 15
```

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter a sequence of space-separated numbers:
67 65 68 70 69
Error: Expected exactly two missing numbers.
```

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter a sequence of space-separated numbers:
88 87 85 86 90 89
Error: Expected exactly two missing numbers.
```

## Question 2: Zigzag Array [15 marks]

Write a Python program called `zigzag.py` that rearranges an unsorted array such that the output alternates between **low, high, low, high …** when read left to right.

For example, consider the array:

```
7 3 5 2 9 1
Sorted list: [1,2,3,5,7,9]
Zigzag output: [1,9,2,7,3,5]
```

You must sort manually using any algorithm (e.g., selection sort) – see ***Appendix A*** *at the end of the assignment sheet*.

**You may not use any built-in functions to answer this question.** However, you may use `input()`, `print()`, `list()`, `map()`, `strip()`, `append()`.

HINT:

- Use two pointers:
  - One at the start (low) for the smallest element.
  - One at the end (high) for the largest element.
- Sort the array into ascending order first.
- Alternate between picking the largest and smallest elements while constructing the new order.

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter numbers a sequence of space-separated numbers:
7 3 5 2 9 1 6 4
Zigzag array: [1, 9, 2, 7, 3, 6, 4, 5]
```

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter numbers a sequence of space-separated numbers:
12 15 22
Zigzag array: [12, 22, 15]
```

## Question 3: Matrix Sum [20 marks]

Write a program called `matrix_sum.py` that:

1. Prompts the user to enter a **4×4 grid** of integers (4 lines, each containing 4 digits without spaces, e.g., "4581").
2. Constructs a 2D list from the input.
3. Asks the user for a coordinate (row, col) and prints the sum of the values in the **row** and **column** of that position.
4. Entering -1 -1 ends the program.

Follow the sample IO below for a detailed example.

**You may not use any built-in functions to answer this question.** However, you may use `input()`, `print()`, `int()`, `strip()`, `append()`.

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter 4 rows (each 4 digits, e.g., 2941):
4679
9824
6672
1650
Enter coordinates: 3 3
Row sum = 12
Column sum = 15
Enter coordinates: -1 -1
```

***Sample IO*** *(The input from the user is shown in* ***bold*** *font – do not program this):*

```
Enter 4 rows (each 4 digits, e.g., 2941):
4679
9824
6672
1650
Enter coordinates: 0 0
Row sum = 26
Column sum = 20
Enter coordinates: 1 3
Row sum = 23
Column sum = 15
Enter coordinates: 3 2
Row sum = 12
Column sum = 21
Enter coordinates: -1 -1
```

## Submission

Create and submit a Zip file called '`ABCXYZ123.zip`' (where ABCXYZ123 is YOUR student number) containing `second_missing.py`, `zigzag.py` and `matrix_sum.py`.

## Appendix A: Selection Sort

Consider the following unsorted list:

**34 17 12 34 55 48 23**

The selection sort algorithm works by repeatedly finding the **minimum value** from the unsorted portion and swapping it with the element at the current position. A step-by-step selection sort algorithm on the above list is as follows:

***Pass 1 (***`i = 0`***)***
Unsorted portion: **34, 17, 12, 34, 55, 48, 23**
Minimum value = **12**
Swap position 0 with position 2.
→ **[12, 17, 34, 34, 55, 48, 23]**

***Pass 2 (***`i = 1`***)***
Unsorted portion: **17, 34, 34, 55, 48, 23**
Minimum value = **17**
Already in correct place → no swap.
→ **[12, 17, 34, 34, 55, 48, 23]**

***Pass 3 (***`i = 2`***)***
Unsorted portion: **34, 34, 55, 48, 23**
Minimum value = **23**
Swap position 2 with position 6.
→ **[12, 17, 23, 34, 34, 55, 48]**

***Pass 4 (***`i = 3`***)***
Unsorted portion: **34, 34, 55, 48**
Minimum value = **34**
Already in correct place → no swap.
→ **[12, 17, 23, 34, 34, 55, 48]**

***Pass 5 (***`i = 4`***)***
Unsorted portion: **34, 55, 48**
Minimum value = **34**
Already in correct place → no swap.
→ **[12, 17, 23, 34, 34, 55, 48]**

***Pass 6 (***`i = 5`***)***
Unsorted portion: **55, 48**
Minimum value = **48**
Swap position 5 with position 6.
→ **[12, 17, 23, 34, 34, 48, 55]**

***Pass 7 (***`i = 6`***)***

Only one element left; list is sorted.

**Final Sorted List**

**[12, 17, 23, 34, 34, 48, 55**

The algorithm:

```
def selection_sort ( values ):
    """Sort values using selection sort algorithm."""
    # iterate over outer positions in list
    for outer in range (len (values)):
        # assume first value is minimum
        minimum = outer
        # compare minimum to rest of list and update
        for inner in range (outer+1, len (values)):
            if values[inner] < values[minimum]:
                minimum = inner
        # swap minimum with outer position
        values[minimum],values[outer]= values[outer],values[minimum]
```